FLUTTER LAB SHEET – 9

REST API Integration and JSON Parsing

Complete solutions for all 15 programs

Setup Instructions

Add the HTTP package under dependencies in pubspec.yaml, then run flutter pub get. Use one program at a time in lib/main.dart. Programs 1 and 2 are standalone Dart console programs.

```
dependencies:\n  flutter:\n    sdk: flutter\n  http: ^1.2.2
```

The examples use JSONPlaceholder and Fake Store API demo endpoints. An internet connection is required. The demo POST endpoint returns a sample response and does not permanently store the submitted post.

1. Parse JSON array into Student objects

```
import 'dart:convert';

class Student {
  final int id;
  final String name, email;
  Student({required this.id, required this.name, required this.email});

  factory Student.fromJson(Map<String, dynamic> j) =>
      Student(id: j['id'], name: j['name'], email: j['email']);
}

void main() {
  const data = '[{"id":1,"name":"Aarav","email":"aarav@mail.com"},'
      '{"id":2,"name":"Meera","email":"meera@mail.com"}]';
  final List list = jsonDecode(data);
  final students = list.map((e) => Student.fromJson(e)).toList();
  for (final s in students) {
    print('${s.id} ${s.name} ${s.email}');
  }
}
```

2. Demonstrate Future, async and await

```
Future<String> loadMessage() async {
  await Future.delayed(const Duration(seconds: 2));
  return 'Data received successfully';
}

Future<void> main() async {
  print('Loading...');
  final result = await loadMessage();
  print(result);
}
```

3. Fetch API response using http.get()

```
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

void main() => runApp(const MaterialApp(home: ApiPage()));

class ApiPage extends StatefulWidget {
  const ApiPage({super.key});
  State<ApiPage> createState() => _ApiPageState();
}
class _ApiPageState extends State<ApiPage> {
  String result = 'Press Fetch Data';
  Future<void> fetch() async {
    try {
      final r = await http.get(Uri.parse('https://jsonplaceholder.typicode.com/posts/1'));
      setState(() => result = r.body);
    } catch (e) { setState(() => result = 'Error: $e'); }
  }
  Widget build(BuildContext c) => Scaffold(
    appBar: AppBar(title: const Text('REST API')),
    body: Padding(padding: const EdgeInsets.all(16), child: Column(children: [
      ElevatedButton(onPressed: fetch, child: const Text('Fetch Data')),
      Expanded(child: SingleChildScrollView(child: Text(result)))
    ])));
}
```

4. Fetch one post and display it in Card

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

void main() => runApp(const MaterialApp(home: PostPage()));
class PostPage extends StatefulWidget {
  const PostPage({super.key});
  State<PostPage> createState() => _PostPageState();
}
class _PostPageState extends State<PostPage> {
  Map? post; bool loading = true;
  @override void initState() { super.initState(); fetch(); }
  Future<void> fetch() async {
    try {
      final r = await http.get(Uri.parse('https://jsonplaceholder.typicode.com/posts/1'));
      if (r.statusCode == 200) setState(() => post = jsonDecode(r.body));
    } finally { setState(() => loading = false); }
  }
  Widget build(BuildContext c) => Scaffold(
    appBar: AppBar(title: const Text('Single Post')),
    body: Center(child: loading ? const CircularProgressIndicator() :
      post == null ? const Text('Unable to load') : Card(
        margin: const EdgeInsets.all(16), child: Padding(
          padding: const EdgeInsets.all(16), child: Column(mainAxisSize: MainAxisSize.min,
          children: [Text(post!['title']), const SizedBox(height: 10), Text(post!['body'])])))));
}
```

5. Show CircularProgressIndicator while fetching

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

void main() => runApp(const MaterialApp(home: LoadingPage()));
class LoadingPage extends StatefulWidget {
  const LoadingPage({super.key});
  State<LoadingPage> createState() => _LoadingPageState();
}
class _LoadingPageState extends State<LoadingPage> {
  String? title;
  @override void initState() { super.initState(); fetch(); }
  Future<void> fetch() async {
    final r = await http.get(Uri.parse('https://jsonplaceholder.typicode.com/posts/1'));
    if (!mounted) return;
    if (r.statusCode == 200) setState(() => title = jsonDecode(r.body)['title']);
    else setState(() => title = 'Request failed');
  }
  Widget build(BuildContext c) => Scaffold(
    appBar: AppBar(title: const Text('Loading')),
    body: Center(child: title == null ? const CircularProgressIndicator() : Text(title!)));
}
```

6. Display API data using FutureBuilder

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

Future<Map<String,dynamic>> getPost() async {
  final r = await http.get(Uri.parse('https://jsonplaceholder.typicode.com/posts/1'));
  if (r.statusCode != 200) throw Exception('Failed to load');
  return jsonDecode(r.body);
}
void main() => runApp(const MaterialApp(home: FuturePage()));
class FuturePage extends StatelessWidget {
  const FuturePage({super.key});
  Widget build(BuildContext c) => Scaffold(
    appBar: AppBar(title: const Text('FutureBuilder')),
    body: FutureBuilder(future: getPost(), builder: (c, snap) {
      if (snap.connectionState == ConnectionState.waiting)
        return const Center(child: CircularProgressIndicator());
      if (snap.hasError) return Center(child: Text('Error: ${snap.error}'));
      final p = snap.data as Map<String,dynamic>;
      return Padding(padding: const EdgeInsets.all(16),
        child: Column(children: [Text(p['title']), Text(p['body'])]));
    }));
}
```

7. Fetch post titles using ListView.builder

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

void main() => runApp(const MaterialApp(home: PostsPage()));
class PostsPage extends StatefulWidget {
  const PostsPage({super.key});
  State<PostsPage> createState() => _PostsPageState();
}
class _PostsPageState extends State<PostsPage> {
  List posts = []; bool loading = true;
  @override void initState() { super.initState(); fetch(); }
  Future<void> fetch() async {
    final r = await http.get(Uri.parse('https://jsonplaceholder.typicode.com/posts'));
    setState(() { if (r.statusCode == 200) posts = jsonDecode(r.body); loading = false; });
  }
  Widget build(BuildContext c) => Scaffold(
    appBar: AppBar(title: const Text('Post Titles')),
    body: loading ? const Center(child: CircularProgressIndicator()) :
      ListView.builder(itemCount: posts.length, itemBuilder: (c,i) =>
        ListTile(leading: CircleAvatar(child: Text('${i+1}')), title: Text(posts[i]['title']))));
}
```

8. Fetch users with ListTile and CircleAvatar

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

void main() => runApp(const MaterialApp(home: UsersPage()));
class UsersPage extends StatefulWidget {
  const UsersPage({super.key});
  State<UsersPage> createState() => _UsersPageState();
}
class _UsersPageState extends State<UsersPage> {
  List users = []; bool loading = true;
  @override void initState() { super.initState(); fetch(); }
  Future<void> fetch() async {
    final r = await http.get(Uri.parse('https://jsonplaceholder.typicode.com/users'));
    setState(() { if (r.statusCode == 200) users = jsonDecode(r.body); loading = false; });
  }
  Widget build(BuildContext c) => Scaffold(
    appBar: AppBar(title: const Text('Users')),
    body: loading ? const Center(child: CircularProgressIndicator()) :
      ListView.builder(itemCount: users.length, itemBuilder: (c,i) {
        final u = users[i];
        return ListTile(leading: CircleAvatar(child: Text(u['name'][0])),
          title: Text(u['name']), subtitle: Text(u['email']));
      }));
}
```

9. Convert API data into objects using model class

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

class Post {
  final int id; final String title, body;
  Post({required this.id, required this.title, required this.body});
  factory Post.fromJson(Map<String,dynamic> j) =>
    Post(id:j['id'], title:j['title'], body:j['body']);
}
void main() => runApp(const MaterialApp(home: ModelPage()));
class ModelPage extends StatefulWidget {
  const ModelPage({super.key});
  State<ModelPage> createState() => _ModelPageState();
}
class _ModelPageState extends State<ModelPage> {
  List<Post> posts = []; bool loading = true;
  @override void initState() { super.initState(); fetch(); }
  Future<void> fetch() async {
    final r = await http.get(Uri.parse('https://jsonplaceholder.typicode.com/posts'));
    if (r.statusCode == 200) {
      final List data = jsonDecode(r.body);
      setState(() { posts = data.map((e) => Post.fromJson(e)).toList(); loading = false; });
    } else setState(() => loading = false);
  }
  Widget build(BuildContext c) => Scaffold(
    appBar: AppBar(title: const Text('Model Class')),
    body: loading ? const Center(child: CircularProgressIndicator()) :
      ListView(children: posts.map((p) => ListTile(title: Text(p.title), subtitle: Text(p.body))).toList()));
}
```

10. Handle API errors and Retry

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

void main() => runApp(const MaterialApp(home: RetryPage()));
class RetryPage extends StatefulWidget {
  const RetryPage({super.key});
  State<RetryPage> createState() => _RetryPageState();
}
class _RetryPageState extends State<RetryPage> {
  String? title, error; bool loading = false;
  @override void initState() { super.initState(); fetch(); }
  Future<void> fetch() async {
    setState(() { loading = true; error = null; });
    try {
      final r = await http.get(Uri.parse('https://jsonplaceholder.typicode.com/posts/1'))
        .timeout(const Duration(seconds: 10));
      if (r.statusCode != 200) throw Exception('Status ${r.statusCode}');
      setState(() => title = jsonDecode(r.body)['title']);
    } catch (_) { setState(() => error = 'No internet or invalid response.'); }
    finally { setState(() => loading = false); }
  }
  Widget build(BuildContext c) => Scaffold(
    appBar: AppBar(title: const Text('Error Handling')),
    body: Center(child: loading ? const CircularProgressIndicator() :
      error != null ? Column(mainAxisSize: MainAxisSize.min, children: [
        Text(error!), ElevatedButton(onPressed: fetch, child: const Text('Retry'))]) : Text(title ?? '')));
}
```

11. User list and detail screen

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

void main() => runApp(const MaterialApp(home: UserListPage()));
class UserListPage extends StatefulWidget {
  const UserListPage({super.key});
  State<UserListPage> createState() => _UserListPageState();
}
class _UserListPageState extends State<UserListPage> {
  List users = []; bool loading = true;
  @override void initState() { super.initState(); fetch(); }
  Future<void> fetch() async {
    final r = await http.get(Uri.parse('https://jsonplaceholder.typicode.com/users'));
    setState(() { if (r.statusCode == 200) users = jsonDecode(r.body); loading = false; });
  }
  Widget build(BuildContext c) => Scaffold(
    appBar: AppBar(title: const Text('User List')),
    body: loading ? const Center(child: CircularProgressIndicator()) :
      ListView.builder(itemCount: users.length, itemBuilder: (c,i) {
        final u = users[i];
        return ListTile(title: Text(u['name']), subtitle: Text(u['email']),
          onTap: () => Navigator.push(c, MaterialPageRoute(builder: (_) => UserDetail(user:u))));
      }));
}
class UserDetail extends StatelessWidget {
  final Map user;
  const UserDetail({super.key, required this.user});
  Widget build(BuildContext c) => Scaffold(
    appBar: AppBar(title: const Text('User Details')),
    body: Padding(padding: const EdgeInsets.all(16), child: Column(
      crossAxisAlignment: CrossAxisAlignment.start, children: [
        Text('Name: ${user['name']}'), Text('Username: ${user['username']}'),
        Text('Email: ${user['email']}'), Text('Phone: ${user['phone']}'),
        Text('Website: ${user['website']}'), Text('Company: ${user['company']['name']}'),
        Text('City: ${user['address']['city']}')
      ])));
}
```

12. Filter API data using Search TextField

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

void main() => runApp(const MaterialApp(home: SearchPage()));
class SearchPage extends StatefulWidget {
  const SearchPage({super.key});
  State<SearchPage> createState() => _SearchPageState();
}
class _SearchPageState extends State<SearchPage> {
  List users = []; bool loading = true; String query = '';
  @override void initState() { super.initState(); fetch(); }
  Future<void> fetch() async {
    final r = await http.get(Uri.parse('https://jsonplaceholder.typicode.com/users'));
    setState(() { if (r.statusCode == 200) users = jsonDecode(r.body); loading = false; });
  }
  Widget build(BuildContext c) {
    final filtered = users.where((u) =>
      u['name'].toString().toLowerCase().contains(query.toLowerCase()) ||
      u['email'].toString().toLowerCase().contains(query.toLowerCase())).toList();
    return Scaffold(appBar: AppBar(title: const Text('Search Users')),
      body: loading ? const Center(child: CircularProgressIndicator()) : Column(children: [
```

```
        Padding(padding: const EdgeInsets.all(12), child: TextField(
          decoration: const InputDecoration(labelText: 'Search name or email',
            prefixIcon: Icon(Icons.search), border: OutlineInputBorder()),
          onChanged: (v) => setState(() => query = v))),
        Expanded(child: ListView.builder(itemCount: filtered.length, itemBuilder: (c,i) =>
          ListTile(title: Text(filtered[i]['name']), subtitle: Text(filtered[i]['email']))))
      ]));
  }
}
```

13. Reload API data using RefreshIndicator

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

void main() => runApp(const MaterialApp(home: RefreshPage()));
class RefreshPage extends StatefulWidget {
  const RefreshPage({super.key});
  State<RefreshPage> createState() => _RefreshPageState();
}
class _RefreshPageState extends State<RefreshPage> {
  List posts = []; bool loading = true;
  @override void initState() { super.initState(); fetch(); }
  Future<void> fetch() async {
    final r = await http.get(Uri.parse('https://jsonplaceholder.typicode.com/posts'));
    if (mounted) setState(() { if (r.statusCode == 200) posts = jsonDecode(r.body); loading = false; });
  }
  Widget build(BuildContext c) => Scaffold(
    appBar: AppBar(title: const Text('Pull to Refresh')),
    body: loading ? const Center(child: CircularProgressIndicator()) :
      RefreshIndicator(onRefresh: fetch, child: ListView.builder(
        physics: const AlwaysScrollableScrollPhysics(), itemCount: posts.length,
        itemBuilder: (c,i) => ListTile(title: Text(posts[i]['title'])))));
}
```

14. Send data using http.post() from a form

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

void main() => runApp(const MaterialApp(home: CreatePostPage()));
class CreatePostPage extends StatefulWidget {
  const CreatePostPage({super.key});
  State<CreatePostPage> createState() => _CreatePostPageState();
}
class _CreatePostPageState extends State<CreatePostPage> {
  final title = TextEditingController(), body = TextEditingController();
  String message = ''; bool sending = false;
  Future<void> submit() async {
    setState(() { sending = true; message = ''; });
    try {
      final r = await http.post(Uri.parse('https://jsonplaceholder.typicode.com/posts'),
        headers: {'Content-Type':'application/json; charset=UTF-8'},
        body: jsonEncode({'title':title.text,'body':body.text,'userId':1}));
      setState(() => message = r.statusCode == 201
        ? 'Post created with ID ${jsonDecode(r.body)['id']}' : 'Failed: ${r.statusCode}');
    } catch (e) { setState(() => message = 'Error: $e'); }
    finally { setState(() => sending = false); }
  }
  void dispose() { title.dispose(); body.dispose(); super.dispose(); }
  Widget build(BuildContext c) => Scaffold(appBar: AppBar(title: const Text('Create Post')),
    body: Padding(padding: const EdgeInsets.all(16), child: Column(children: [
      TextField(controller:title, decoration: const InputDecoration(labelText:'Title')),
      TextField(controller:body, decoration: const InputDecoration(labelText:'Body')),
      ElevatedButton(onPressed:sending ? null : submit, child: Text(sending ? 'Sending...' : 'Submit')),
      Text(message)
    ])));
}
```

15. Product Store with GridView and detail screen

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

void main() => runApp(const MaterialApp(home: ProductStore()));
class ProductStore extends StatefulWidget {
  const ProductStore({super.key});
  State<ProductStore> createState() => _ProductStoreState();
}
class _ProductStoreState extends State<ProductStore> {
  List products = []; bool loading = true; String? error;
  @override void initState() { super.initState(); fetch(); }
  Future<void> fetch() async {
    setState(() { loading = true; error = null; });
    try {
      final r = await http.get(Uri.parse('https://fakestoreapi.com/products'));
      if (r.statusCode != 200) throw Exception('Status ${r.statusCode}');
      setState(() { products = jsonDecode(r.body); loading = false; });
    } catch (e) { setState(() { error = 'Unable to load products: $e'; loading = false; }); }
  }
  Widget build(BuildContext c) => Scaffold(appBar: AppBar(title: const Text('Product Store')),
    body: loading ? const Center(child: CircularProgressIndicator()) :
      error != null ? Center(child: Column(mainAxisSize: MainAxisSize.min, children: [
        Text(error!), ElevatedButton(onPressed: fetch, child: const Text('Retry'))])) :
      GridView.builder(padding: const EdgeInsets.all(10),
        gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
          crossAxisCount:2, childAspectRatio:.68, crossAxisSpacing:10, mainAxisSpacing:10),
        itemCount:products.length, itemBuilder:(c,i) {
          final p = products[i];
          return InkWell(onTap:()=>Navigator.push(c, MaterialPageRoute(
            builder:(_) => ProductDetail(product:p))),
            child: Card(child: Padding(padding:const EdgeInsets.all(8), child:Column(children:[
              Expanded(child:Image.network(p['image'], fit:BoxFit.contain,
                errorBuilder:(_,__,___)=>const Icon(Icons.image_not_supported))),
              Text(p['title'],maxLines:2,overflow:TextOverflow.ellipsis),
              Text('\$${p['price']}')
            ])));
        }));
}
class ProductDetail extends StatelessWidget {
  final Map product;
  const ProductDetail({super.key, required this.product});
  Widget build(BuildContext c) => Scaffold(appBar:AppBar(title:const Text('Product Details')),
    body:Padding(padding:const EdgeInsets.all(16), child:ListView(children:[
      SizedBox(height:220, child:Image.network(product['image'],fit:BoxFit.contain)),
      Text(product['title'],style:Theme.of(c).textTheme.titleLarge),
      Text('\$${product['price']}'),
      Text(product['description']),
      Text('Category: ${product['category']}')
    ])));
}
```